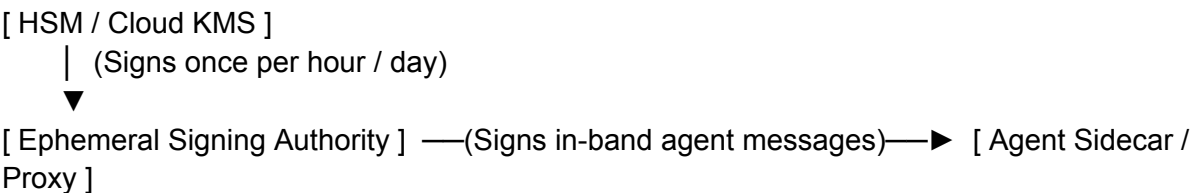


Securing agent control plane signing keys requires isolating private keys within a Hardware Security Module (HSM, FIPS 140-3 Level 3/4) or a cloud Key Management Service (KMS), ensuring keys remain non-exportable and inaccessible to agent runtimes or host operating systems. The control plane delegates signature generation via authenticated RPCs (such as PKCS#11, KMIP, or Cloud KMS APIs), ensuring that a compromised orchestrator instance cannot leak root private keys.

1. Two-Tier Key Hierarchy Architecture

Directly invoking an HSM or KMS for high-frequency inter-agent control messages introduces latency and rate-limiting bottlenecks. To maintain millisecond-level responsiveness across large swarms, implement an ephemeral key delegation pattern:



- **Root Control Key (HSM/KMS Boundary):** A long-lived, non-exportable asymmetric key pair (Ed25519 or ECDSA P-256). Its sole role is signing short-lived signing certificates or JSON Web Key Sets (JWKS) for control workers.
- **Ephemeral Worker Keys (Local Vault / In-Memory):** Short-lived key pairs generated in volatile memory with explicit Time-To-Live (TTL: 15–60 minutes). Used by orchestrator nodes to sign real-time agent control messages ($O(1)$ latency).

2. HSM/KMS Integration Mechanics

Phase	Component	Operational Mechanism
Bootstrapping	HSM / KMS	Generates root key pair (PK_{root} , SK_{root}) with immutable policy attributes ($CKA_EXTRACTABLE = FALSE$, $CKA_NEVER_EXTRACTABLE = TRUE$).
Delegation	Control Orchestrator	Generates local short-lived key pair ($PK_{\text{ephemeral}}$, $SK_{\text{ephemeral}}$). Constructs a Certificate Signing Request (CSR) and calls

		<code>kms:sign</code> to issue an operational intermediate cert.
Signing	Orchestrator Worker	Signs in-band agent control payloads using <code>\$SK_{\text{ephemeral}}\$</code> . Attaches key ID (<code>kid</code>) and intermediate certificate chain to the message envelope.
Verification	Ingress Proxy / Sidecar	Fetches and caches <code>\$PK_{\text{root}}\$</code> from an out-of-band JWKS endpoint. Verifies the message signature against <code>\$PK_{\text{ephemeral}}\$</code> and validates that <code>\$PK_{\text{ephemeral}}\$</code> is signed by <code>\$PK_{\text{root}}\$</code> .

3. Access Control & Operational Security

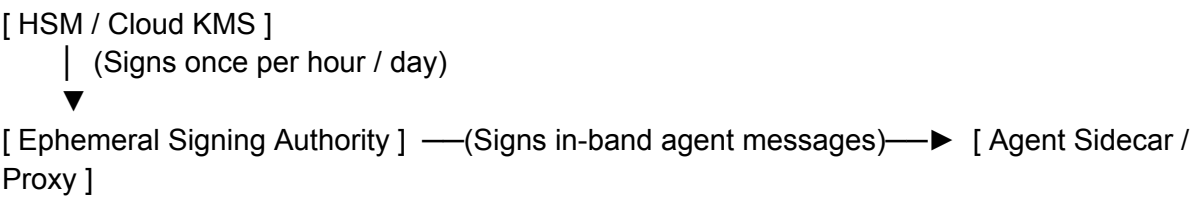
- **Workload Identity Attestation:** Bind HSM/KMS signing permissions strictly to workload identity providers (e.g., SPIFFE/SPIRE SVIDs or AWS IRSA / GCP Workload Identity). Require cryptographic proof of orchestrator binary integrity (attestation via TPM 2.0 / Confidential Computing) before granting KMS signing access.
- **Least-Privilege Policy Scopes:** Restrict orchestrator IAM roles exclusively to `kms:Sign` and `kms:GetPublicKey`. Block access to administrative management, decryption, or key deletion endpoints.
- **Audit Logging & Automated Freeze:** Stream HSM audit trails directly into security monitoring systems. An anomalous spike in `kms:Sign` requests triggers an automated freeze on new ephemeral key generation, falling back to emergency quarantine protocols.

Introducing SPIFFE for Agents

Securing agent control plane signing keys requires isolating private keys within a Hardware Security Module (HSM, FIPS 140-3 Level 3/4) or a cloud Key Management Service (KMS), ensuring keys remain non-exportable and inaccessible to agent runtimes or host operating systems. The control plane delegates signature generation via authenticated RPCs (such as PKCS#11, KMIP, or Cloud KMS APIs), ensuring that a compromised orchestrator instance cannot leak root private keys.

1. Two-Tier Key Hierarchy Architecture

Directly invoking an HSM or KMS for high-frequency inter-agent control messages introduces latency and rate-limiting bottlenecks. To maintain millisecond-level responsiveness across large swarms, implement an ephemeral key delegation pattern:



- **Root Control Key (HSM/KMS Boundary):** A long-lived, non-exportable asymmetric key pair (Ed25519 or ECDSA P-256). Its sole role is signing short-lived signing certificates or JSON Web Key Sets (JWKS) for control workers.
- **Ephemeral Worker Keys (Local Vault / In-Memory):** Short-lived key pairs generated in volatile memory with explicit Time-To-Live (TTL: 15–60 minutes). Used by orchestrator nodes to sign real-time agent control messages ($O(1)$ latency).

2. HSM/KMS Integration Mechanics

Phase	Component	Operational Mechanism
Bootstrapping	HSM / KMS	Generates root key pair (PK_{root} , SK_{root}) with immutable policy attributes ($CKA_EXTRACTABLE = FALSE$, $CKA_NEVER_EXTRACTABLE = TRUE$).

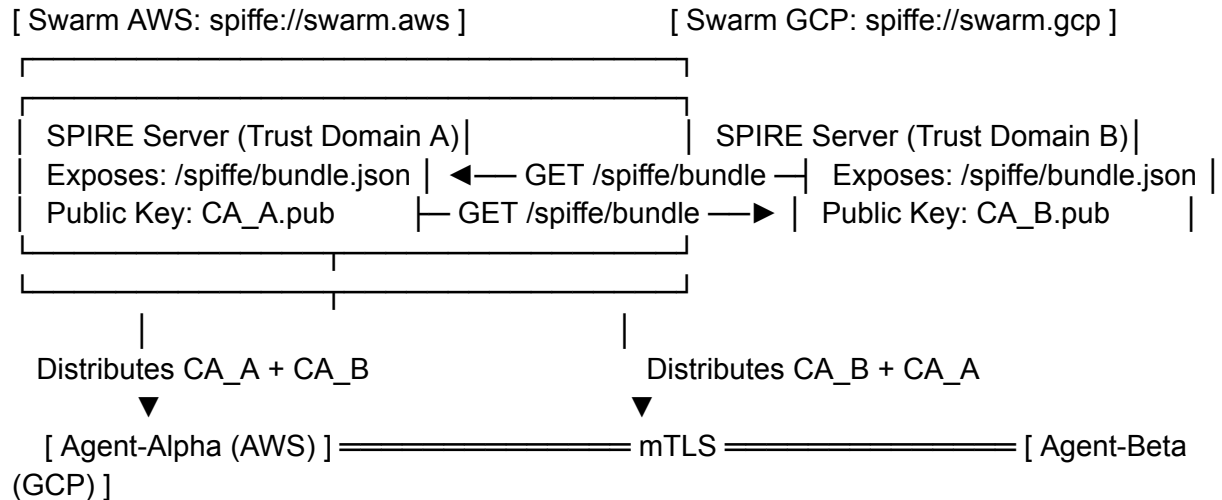
Delegation	Control Orchestrator	Generates local short-lived key pair (\$PK_{\text{ephemeral}}, SK_{\text{ephemeral}}\$). Constructs a Certificate Signing Request (CSR) and calls <code>kms:sign</code> to issue an operational intermediate cert.
Signing	Orchestrator Worker	Signs in-band agent control payloads using \$SK_{\text{ephemeral}}\$. Attaches key ID (<code>kid</code>) and intermediate certificate chain to the message envelope.
Verification	Ingress Proxy / Sidecar	Fetches and caches \$PK_{\text{root}}\$ from an out-of-band JWKS endpoint. Verifies the message signature against \$PK_{\text{ephemeral}}\$ and validates that \$PK_{\text{ephemeral}}\$ is signed by \$PK_{\text{root}}\$.

3. Access Control & Operational Security

- **Workload Identity Attestation:** Bind HSM/KMS signing permissions strictly to workload identity providers (e.g., SPIFFE/SPIRE SVIDs or AWS IRSA / GCP Workload Identity). Require cryptographic proof of orchestrator binary integrity (attestation via TPM 2.0 / Confidential Computing) before granting KMS signing access.
- **Least-Privilege Policy Scopes:** Restrict orchestrator IAM roles exclusively to `kms:Sign` and `kms:GetPublicKey`. Block access to administrative management, decryption, or key deletion endpoints.
- **Audit Logging & Automated Freeze:** Stream HSM audit trails directly into security monitoring systems. An anomalous spike in `kms:Sign` requests triggers an automated freeze on new ephemeral key generation, falling back to emergency quarantine protocols.

SPIFFE Federation enables autonomous agent swarms in separate cloud environments or trust domains (e.g., `spiffe://swarm.aws` and `spiffe://swarm.gcp`) to establish cryptographic trust without sharing a root CA or exposing private keys. SPIRE Servers continuously synchronize public key material (**Trust Bundles**) over authenticated HTTPS **Bundle Endpoints**, allowing workloads in one domain to validate X.509-SVIDs presented by workloads in another.

Federation Architecture & Bundle Exchange



1. **Bundle Endpoint Exposure:** Each trust domain's SPIRE Server exposes a public or inter-vpc endpoint serving its active public CA certificates as a RFC 7517 JSON Web Key Set (JWKS).
2. **Out-of-Band Sync:** SPIRE Servers poll each other's bundle endpoints using the [https_spiffe](#) (mutual SPIFFE authentication) or [https_web](#) (standard Web PKI TLS) profile to fetch remote root keys.
3. **Workload Bundle Push:** When an agent workload is registered with a [federates_with](#) policy, the local SPIRE Agent pushes the foreign trust bundle into the workload's memory via the SPIFFE Workload API.

Step-by-Step Configuration

Step 1: Expose the SPIFFE Bundle Endpoint on SPIRE Server A

On [spiffe://swarm.aws](#), update [spire-server.conf](#) to expose its bundle endpoint and enable the Federation API:

```
Terraform
server {
  bind_address = "0.0.0.0"
  bind_port    = "8081"
  trust_domain = "swarm.aws"

  # Enable the SPIFFE Federation Bundle Endpoint
  federation {
    bundle_endpoint {
      address = "0.0.0.0"
      port    = 8443
      # Profile options: "https_spiffe" (SPIFFE mTLS) or "https_web" (Web PKI)
      acme {
        domain_name = "spire.aws.swarms.internal"
        email        = "admin@swarms.internal"
      }
    }
  }
}
```

```

    }
  }
}
}

```

Step 2: Configure Federation Relationship on SPIRE Server B

On `spiffe://swarm.gcp`, configure `spire-server.conf` to federate with `swarm.aws`:

Terraform

```

server {
  trust_domain = "swarm.gcp"

  federation {
    federates_with "swarm.aws" {
      bundle_endpoint_url = "https://spire.aws.swarms.internal:8443"
      bundle_endpoint_profile "https_web" {}
    }
  }
}

```

(Alternatively, in Kubernetes environments, apply a *SPIREFederatedTrustDomain* CRD resource on the target cluster).

Step 3: Register Agents with Federated Access

To allow **Agent-Beta** on GCP to accept connections from **Agent-Alpha** on AWS, register Agent-Beta's SPIRE entry with explicit access to the `swarm.aws` trust domain:

Bash

```

spire-server entry create \
  -spiffeID spiffe://swarm.gcp/ns/agent-mesh/sa/agent-beta \
  -parentID spiffe://swarm.gcp/spire/agent/k8s-sat/cluster-gcp \
  -selector k8s:ns:agent-mesh \
  -selector k8s:sa:agent-beta-sa \
  -federatesWith "spiffe://swarm.aws"

```

Verification & Cross-Swarm mTLS Execution

Once federated, the SPIFFE Workload API running on GCP provides Agent-Beta with a dual-trust bundle containing both `CA_gcp.pub` and `CA_aws.pub`.

When Agent-Alpha (`spiffe://swarm.aws/ns/agent-mesh/sa/agent-alpha`) initiates a cross-cloud REST or gRPC call to Agent-Beta:

1. **TLS Handshake:** Agent-Alpha presents an SVID signed by `CA_aws.priv`.

2. **Cross-Validation:** Agent-Beta's TLS stack inspects the incoming certificate issuer (`swarm.aws`) and validates the signature against the federated `CA_aws.pub` bundle synced from SPIRE Server A.
3. **Application Authorization:** The agent framework verifies the SAN URI matches expected identity policies (e.g., verifying `spiffe://swarm.aws/...` has permissions to invoke local inference tools).

While X.509-SVIDs provide layer-4 mTLS security, **JWT-SVIDs** allow agents to authenticate statelessly over Layer-7 HTTP/gRPC protocols or federate identity with third-party APIs (e.g., AWS STS, Vault, or external vendor APIs). The **SPIRE OIDC Discovery Provider** bridges the SPIFFE control plane to standard OpenID Connect by serving an OIDC discovery document (`/well-known/openid-configuration`) and a public JSON Web Key Set (JWKS) containing the active public keys needed to verify signed agent tokens.

1. SPIRE Server Configuration

Configure `spire-server.conf` to declare the public canonical issuer URL injected into every generated token's `iss` claim:

```
Terraform
server {
  trust_domain = "swarm.internal"
  jwt_issuer   = "https://oidc.swarm.internal"

  # Default TTL for short-lived inter-agent tokens (e.g., 5m to 1h)
  default_jwt_svid_ttl = "15m"
}
```

2. Configuring the SPIRE OIDC Discovery Provider

Deploy the `oidc-discovery-provider` service fronted by an ingress controller or TLS reverse proxy. Configure its HCL file (`oidc-discovery-provider.conf`) to pull active public key material directly from SPIRE Server APIs:

```
Terraform
log_level = "info"
domains   = ["oidc.swarm.internal"]

# Listen on a local Unix socket behind an Envoy/Nginx proxy
listen_socket_path = "/run/oidc-discovery-provider/oidc.sock"

# Connect to SPIRE Server API to fetch public JWT keybundles
server_api {
  address = "spire-server.agent-mesh.svc.cluster.local:8081"
}
```

This helper exposes standard OIDC discovery endpoints:

- [GET /.well-known/openid-configuration](#) — Returns discovery metadata.
- [GET /keys](#) (or [/.well-known/jwks.json](#)) — Exposes active public RSA/ECDSA verification keys.

3. Agent JWT-SVID Token Issuance

When **Agent-A** initiates a Layer-7 call to **Agent-B**, Agent-A requests an audience-scoped JWT-SVID from its local SPIRE Workload API socket:

Python

```
from pyspiffe.workloadapi import WorkloadApiClient
```

```
with WorkloadApiClient() as client:
```

```
    # Fetch a JWT-SVID explicitly bound to Agent-B's service audience
```

```
    jwt_svid = client.fetch_jwt_svid(  
        audiences={"https://agent-b.swarm.internal/v1"}  
    )
```

```
    bearer_token = jwt_svid.token
```

4. Verification & Claim Enforcement

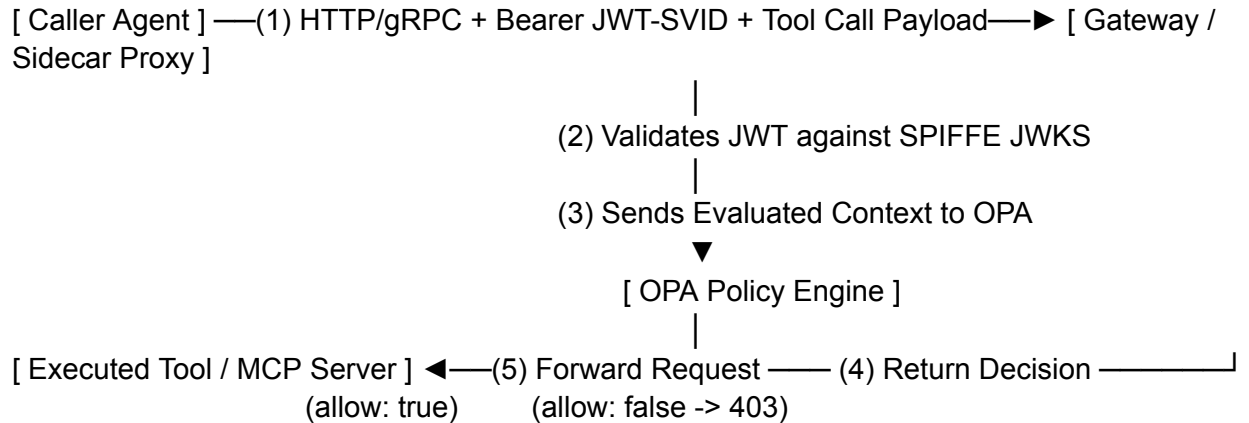
Upon receiving [Authorization: Bearer <token>](#), **Agent-B** validates the JWT using standard OIDC middleware against [\[https://oidc.swarm.internal/keys\]\(https://oidc.swarm.internal/keys\)](#):

- **Header Validation:** Confirms cryptographic algorithm ([alg: RS256](#) or [ES256](#)) and matching Key ID ([kid](#)).
- **Claim Assertions:**
 - [iss](#): Must strictly match ["\[https://oidc.swarm.internal\]\(https://oidc.swarm.internal\)"](#).
 - [aud](#): Must match Agent-B's service URI (["\[https://agent-b.swarm.internal/v1\]\(https://agent-b.swarm.internal/v1\)"](#)).
 - [sub](#): Evaluates the calling agent's unique SPIFFE ID (e.g., ["spiffe://swarm.internal/ns/agent-mesh/sa/agent-a"](#)).
 - [exp](#) / [iat](#): Ensures token freshness and rejects expired credentials.

Combining **SPIFFE JWT-SVID authentication (AuthN)** with **Open Policy Agent (OPA) fine-grained authorization (AuthZ)** decouples identity verification from business logic and safety rules.

In an autonomous agent network, SPIFFE guarantees *who* the calling agent is (its verified cryptographic identity), while OPA determines *whether* that specific agent is allowed to execute a particular tool call given the current request payload, execution depth, and runtime parameters.

1. Architectural Flow



1. **Authentication:** The sidecar/proxy intercepts the tool invocation request, validates the bearer JWT-SVID signature using the SPIFFE OIDC JWKS endpoint, and extracts claims (e.g., `sub: spiffe://swarm.internal/ns/agent-mesh/sa/data-analyst`).
2. **Policy Context Assembly:** The proxy formats an JSON input object containing the caller's validated SPIFFE identity, target tool metadata, and call arguments.
3. **OPA Decision:** OPA evaluates the input against declarative Rego rules and returns a binary `allow` decision along with contextual constraints.

2. OPA Input Context Payload

When an agent attempts to invoke a tool (e.g., `execute_sql_query`), the proxy submits this JSON structure to OPA's `/v1/data/agent/tools/allow` endpoint:

```

JSON
{
  "input": {
    "identity": {
      "spiffe_id": "spiffe://swarm.internal/ns/agent-mesh/sa/data-analyst",
      "trust_domain": "swarm.internal",
      "issuer": "https://oidc.swarm.internal"
    },
    "request": {
      "tool_name": "execute_sql_query",
      "action": "READ",
      "target_resource": "db.analytics.sales_data",
      "parameters": {
        "query": "SELECT * FROM sales_data WHERE year = 2026;",
        "row_limit": 500
      }
    },
    "execution_depth": 2
  }
}

```

3. Fine-Grained Rego Policy

This Rego policy enforces role-based tool access, strict input parameter limits, and maximum delegation depth bounds:

Code snippet

```
package agent.tools
```

```
import future.keywords.in
```

```
# Default deny all tool invocations
```

```
default allow = false
```

```
# Allow decision evaluated against composite safety rules
```

```
allow {
```

```
    valid_spiffe_identity
```

```
    tool_allowed_for_role
```

```
    within_execution_depth
```

```
    valid_parameters
```

```
}
```

```
# Rule 1: Ensure identity belongs to trusted domain and namespace
```

```
valid_spiffe_identity {
```

```
    input.identity.trust_domain == "swarm.internal"
```

```
    startswith(input.identity.spiffe_id, "spiffe://swarm.internal/ns/agent-mesh/")
```

```
}
```

```
# Rule 2: Enforce delegation depth ceiling (prevent recursive sub-agent explosion)
```

```
within_execution_depth {
```

```
    input.request.execution_depth <= 3
```

```
}
```

```
# Rule 3: Match agent role namespace to permitted tool sets
```

```
tool_allowed_for_role {
```

```
    endswith(input.identity.spiffe_id, "/sa/data-analyst")
```

```
    input.request.tool_name in ["execute_sql_query", "generate_chart", "export_csv"]
```

```
}
```

```
tool_allowed_for_role {
```

```
    endswith(input.identity.spiffe_id, "/sa/system-admin")
```

```
    input.request.tool_name in ["execute_sql_query", "restart_service", "update_config"]
```

```
}
```

```
# Rule 4: Dynamic parameter validation (e.g., SQL safety checks)
```

```

valid_parameters {
  input.request.tool_name == "execute_sql_query"
  input.request.parameters.row_limit <= 1000
  not contains_forbidden_keywords(input.request.parameters.query)
}

contains_forbidden_keywords(q) {
  forbidden := ["DROP", "TRUNCATE", "DELETE", "ALTER", "GRANT"]
  some k in forbidden
  contains(upper(q), k)
}

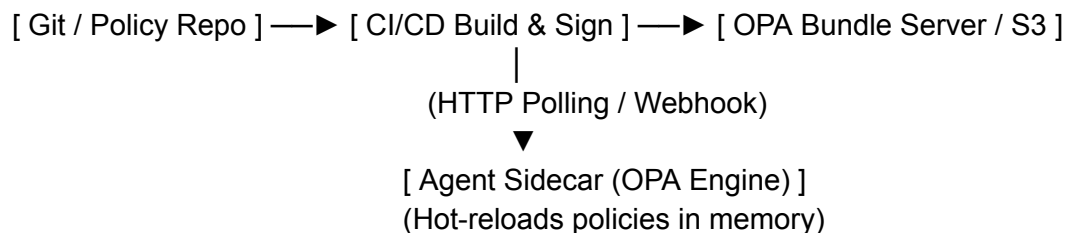
```

4. Policy Decision Matrix

Calling Agent SPIFFE ID	Target Tool	Request Parameter	Execution Depth	OPA Decision	Reason
.../sa/data-analyst	execute_sql_query	row_limit: 500, SELECT...	2	ALLOW	All safety invariants satisfied.
.../sa/data-analyst	execute_sql_query	row_limit: 5000	2	DENY	Exceeds maximum row limit constraint (\$>1000\$).
.../sa/data-analyst	execute_sql_query	DROP TABLE users;	1	DENY	Contains forbidden SQL keyword.
.../sa/data-analyst	restart_service	service: web	1	DENY	Tool not in analyst

					role whitelist.
.../sa/system -admin	restart_service	service: web	4	DENY	Exceeds max delegation depth ceiling (\$>3\$).

An OPA Bundle Server distributes compiled policy sets and context data as gzipped tarballs ([.tar.gz](#)) over standard HTTP/S endpoints, enabling agent sidecars to dynamically hot-reload authorization rules in memory without service restarts or container redeployments.



1. Bundle Build & Cryptographic Signing Pipeline

To ensure agents never execute unvalidated or tampered policies, the CI/CD pipeline tests, compiles, and signs the bundle using an asymmetric private key before publishing it to the bundle storage endpoint.

Bash

1. Run unit tests on Rego policies

```
opa test ./policies/ -v
```

2. Build and sign bundle into a tarball

```
opa build \
  --bundle ./policies/ \
  --signing-key ./keys/policy_signing_private.pem \
  --key-id bundle-key-2026 \
  --output agent-mesh-policy.tar.gz
```

3. Upload signed tarball to bundle server storage (e.g., S3, GCS, or Nginx)

```
aws s3 cp agent-mesh-policy.tar.gz s3://agent-policy-bundles/production/
```

2. OPA Sidecar Bundle Configuration

Each OPA sidecar process running alongside an agent uses an `opa-config.yaml` file that specifies where to fetch bundles, how frequently to poll for updates, and which public key to use for verifying signature payloads.

YAML

```
# /etc/opa/config.yaml
services:
  bundle_server:
    url: https://bundles.swarm.internal/v1
    headers:
      Authorization: "Bearer env:BUNDLE_READ_TOKEN"

bundles:
  agent_security_policies:
    service: bundle_server
    resource: /bundles/agent-mesh-policy.tar.gz
    polling:
      min_delay_seconds: 10
      max_delay_seconds: 30
    signing:
      key: bundle_verification_key

keys:
  bundle_verification_key:
    algorithm: RS256
    public_key: |
      -----BEGIN PUBLIC KEY-----
      MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAuZ...
      -----END PUBLIC KEY-----

# Send policy execution metrics and decision logs back to central storage
decision_logs:
  service: bundle_server
  path: /logs/decisions
  reporting:
    min_delay_seconds: 5
    max_delay_seconds: 15

status:
  service: bundle_server
  path: /status
```

3. Hot-Reloading Mechanics & Runtime Behavior

- **Atomic Swaps in Volatile Memory:** When the OPA sidecar detects a updated `.tar.gz` payload via eTag or HTTP `Last-Modified` headers, it downloads the file, extracts it in isolated memory, and validates the cryptographic signature.
- **Zero-Downtime Swap:** If validation passes, OPA atomically replaces the active pointer in its internal trie structure with the new policy AST (Abstract Syntax Tree). Active agent requests in progress finish on the old policy, while new requests immediately use the updated Rego rules.
- **Automatic Rollback:** If signature verification fails or a newly fetched bundle contains a syntax/parsing error, OPA drops the update, retains its current active policy in memory, and posts a failure payload back to the central `/status` telemetry endpoint.

4. Fleet Management & Telemetry

Feature	Mechanism	Operational Benefit
Status Monitoring	OPA posts JSON heartbeats to <code>/status</code> showing active bundle hash and version.	Ensures 100% visibility into which policy version every agent sidecar is running across the mesh.
Centralized Decision Logging	Asynchronously batches and pushes JSON decision logs to a SIEM / storage bucket.	Provides immutable post-mortem audit trails for all allowed and denied agent tool invocations.
Targeted Distribution	Use distinct bundle paths based on sidecar metadata (<code>/bundles/{region}/{role}.tar.gz</code>).	Delivers role-specific policies to specialized agent clusters instead of bloated monolithic bundles.